

Aufgabe 1: RISC-V Konventionen

1. Mögliche Konventionen

Registersicherung:

1. Register sind unterteilt in: caller-saved vs. callee-saved (wer sichert und restauriert)
2. Gewisse Register dürfen / dürfen nicht vom Unterprogramm verändert werden (saved vs temporary)

Alignment:

(s. 08.93) Daten/Instruktionen dürfen nur auf Adressen abgelegt werden die

1. ein Vielfaches einer Zweierpotenz sind
2. ein Vielfaches der Größe des zugegriffenen Datums sind (natural alignment)

Stack:

1. Stapel wächst von oben nach unten (Stapelzeiger zeigt auf die niedrigste belegt Adresse) oder umgekehrt
2. Parameter liegen auf Stack vor oder nach Rücksprungadresse
3. Stapelzeiger zeigt auf letztes Element oder erste freie Speicheradresse

Parameterübergabe:

1. call-by-value (ein Wert wird übergeben)
2. call-by-reference (eine Referenz wird übergeben. In der Praxis: eine Adresse)
3. call-by-name (ein Ausdruck wird bei Aufruf des Unterprogramms neu ausgewertet)

2. RISC-V Konventionen

Registersicherung:

Register werden grob unterschieden:

1. saved registers `s0` - `s11`: Aufrufer darf vertrauen, dass Unterprogramm diese nicht verändert hat.
2. temporary registers `t0` - `t6`: Unterprogramm darf beliebig überschreiben.
3. Details hängen von genauer Architektur ab (32-Bit, 64-bit, I oder G Extension). Siehe Table 18.2 in RISC-V Calling Convention.

Register	ABI Name	Beschreibung	Saver
x0	zero	Hard-wired Zero	–
x1	ra	Return Address	Caller
x2	sp	Stack Pointer	Callee
x3	gp	Global Pointer	–
x4	tp	Thread Pointer	–
x5-x7	t0-t2	Temporaries	Caller
x8	s0/fp	Saved Register / Frame Pointer	Callee
x9	s1	Saved Register	Callee
x10-x11	a0-a1	Function Arguments / Return Values	Caller
x12-x17	a2-a7	Function Arguments	Caller
x18-x27	s2-s11	Saved Registers	Callee
x28-x31	t3-t6	Temporaries	Caller

Table 1: RISC-V Calling Convention (nach Waterman et al., 2019)

Alignment:

1. Instruction Alignment: Vielfache von 4
2. Data Alignment: Keine Vorgabe, aber Konsequenzen, falls nicht naturally aligned (siehe vorherige Hausaufgabe, A-02c). Konsequenz: natural alignment wird empfohlen.

Stack:

1. Stapel von oben nach unten (Stapelzeiger zeigt auf die niedrigste belegte Adresse)
2. Stack pointer ist auf 16 Bytes aligned
3. Rücksprungadresse auf Stack zu legen ist Verantwortung des non-leaf Unterprogramms
4. Bei Spill in Parameterübergabe zeigt der stack pointer auf das erste auf dem Stack liegende Argument

Parameterübergabe:

Hauptquelle ist RISC-V Calling Convention. *ibid.*: *pointer-word*: register-width, also 32 Bits bei RV32.

1. Parameterübergabe wenn möglich in Registern und by-value.
2. Bei zu vielen Argumenten: Übergabe über Stack.
3. Integer-Register a0-a7 und floating-point Register fa0-fa7
4. Genaue Übergabe hängt wieder von spezifischer Architektur ab
5. Argumente kleiner als ein pointer-word werden in den least significant bits des entsprechenden Registers bzw. in kleineren Adressen auf dem Stack übergeben (wegen little-endian)
6. Argumente größer als doppelte Größe von pointer-word werden by-reference übergeben
7. Ähnliche Regeln für Rückgabe. Werte liegen in a0 (und a1 für 64-Bit Ergebnisse bei RV32).

2. Unterprogramme

2.1 Wie könnte die Implementierung der in der Vorlesung erwähnten Semantiken für Parameter (call-by-value und call-by-reference) in Risc-V aussehen?

In beiden Schemen können Argumente sowohl über Register als auch über den Stack übergeben werden.

Call-by-value

Der Wert wird direkt über eine vereinbarte Methode übergeben:

Per Register:

```
.data
arg0: .word 0x0ca0
arg1: .word 0x0007

.text
j main

# Expect args of integer type in a0-a1 (as per ABI)
add:
# Return value in a0 (as per ABI)
add a0, a0, a1
ret

main:
lw a0, arg0
lw a1, arg1
call add
```

Per Stack:

```
# Pass value via stack
.data
arg0: .word 0x0ca0
arg1: .word 0x0007

.text
j main

# Expect arg(n) at n(sp)
add:
# process data
# load args
lw t0 0 sp
lw t1 4 sp
add a0 t0 t1
ret

main:
lw s0, arg0
lw s1, arg1
# Allocate stack space (simplified--RISC-V ABI demands 128-bit alignment.)
addi sp, sp, -8
sw s1 4 sp
```

```
sw s0 0 sp
call add
# Deallocate stack space
addi sp sp 8
```

Call-by-reference

Die Adresse des Datums wird übergeben:

Per Register:

```
.data
arg0: .word 0x0ca0
arg1: .word 0x0007

.text
j main

add:
lw t0 0 a0 # Load value at [a0]
lw t1 0 a1
add a0, t0, t1
ret

main:
la a0, arg0
la a1, arg1
call add
```

Analog für Stack (siehe oben unter call-by-reference)

Aufgabe 2.2: Division und Rest

Da wir die Befehle `div`, `rem` und `mul` nicht nutzen dürfen, implementieren wir den Algorithmus der **wiederholten Subtraktion** (Repeated Subtraction). Das ist die einfachste Methode: $\frac{a}{b}$ ist die Anzahl, wie oft man b von a abziehen kann, bis der Rest kleiner als b ist.

a) Implementierung in Ripes (Call-by-Value)

Hier werden die Werte direkt in den Registern `a0` und `a1` übergeben.

Dateiname: `div_mod_val.s`

```
.text
.globl main

main:
    # Test: 20 / 3 = 6 Rest 2
    li a0, 20      # Dividend
    li a1, 3       # Divisor

    # Aufruf div(a, b)
    # Argumente sind schon in a0, a1
    # Wir sichern a0/a1, weil div sie verändert
    mv s0, a0
    mv s1, a1

    jal ra, my_div
    # Ergebnis (Quotient) ist jetzt in a0

    # Aufruf mod(a, b)
    mv a0, s0      # Stelle Dividend wieder her
    mv a1, s1      # Stelle Divisor wieder her
    jal ra, my_mod
    # Ergebnis (Rest) ist jetzt in a0

    # Ende
    li a7, 10
    ecall

# -----
# Unterprogramm: my_div
# Eingabe: a0 (Dividend), a1 (Divisor)
# Ausgabe: a0 (Quotient)
# Clobbers: t0
# -----
my_div:
    li t0, 0       # t0 = Counter (Quotient)

    # Sonderfall: Division durch 0 vermeiden (optional)
    beqz a1, div_end

div_loop:
    blt a0, a1, div_end # Wenn a0 < a1, sind wir fertig
    sub a0, a0, a1      # a0 = a0 - a1
    addi t0, t0, 1      # Counter++
    j div_loop

div_end:
    mv a0, t0         # Ergebnis in Return-Register a0
    ret

# -----
# Unterprogramm: my_mod
# Eingabe: a0 (Dividend), a1 (Divisor)
# Ausgabe: a0 (Rest)
# -----
my_mod:
    # Sonderfall: Modulo 0
    beqz a1, mod_end
```

```
mod_loop:
    blt a0, a1, mod_end # Wenn a0 < a1, ist a0 der Rest
    sub a0, a0, a1      # a0 = a0 - a1
    j mod_loop

mod_end:
    ret                # a0 enthält bereits den Rest
```

b) Call-by-Reference

Hier enthalten a0 und a1 nicht die Werte selbst, sondern die **Adressen** im Speicher, wo die Werte liegen. Wir müssen sie erst laden (lw).

Dateiname: div_mod_ref.s

```
.data
val_a: .word 20
val_b: .word 3

.text
.globl main

main:
    la a0, val_a    # Lade ADRESSE von a
    la a1, val_b    # Lade ADRESSE von b

    jal ra, div_ref # Aufruf

    # Ergebnis in a0
    li a7, 10
    ecall

# -----
# Unterprogramm: div_ref
# Eingabe: a0 (Adresse von a), a1 (Adresse von b)
# Ausgabe: a0 (Quotient als Wert)
# -----
div_ref:
    # 1. Werte aus dem Speicher holen (Dereferenzierung)
    lw t0, 0(a0)    # t0 = Wert von a
    lw t1, 0(a1)    # t1 = Wert von b

    li a0, 0        # a0 nutzen wir als Counter/Ergebnis

    beqz t1, ref_end # Check div by zero

ref_loop:
    blt t0, t1, ref_end
    sub t0, t0, t1
    addi a0, a0, 1
    j ref_loop

ref_end:
    ret
```

2.3 Quicksort

Dateiname: quicksort.s

```
.data
lo:
    .word 0xff
    .word 0x17
    .word 0x38
    .word 0x25
    .word 0xa3
    .word 0xc8
    .word 0x84
    .word 0x91
    .word 0x47
    .word 0xb3
    .word 0x91
    .word 0x71
    .word 0xb4
    .word 0xf3
    .word 0x00
hi:
    .word 0x05

.text
j main

swap:
#####
# Swap words [a] with [b]
#
# IN
# a0 = a
# a1 = b
#####
# load values to be exchanged
        lw t0 0 a0
        lw t1 0 a1
# write
        sw t0 0 a1
        sw t1 0 a0
        ret

partition:
#####
# Partition array around pivot using Hoare's method
# pivot = first element
#
# IN
# a0 = start address
# a1 = end address
#
# OUT
# a0 = pivot address
#
# REGISTER MAP
# s0 = i (addr)
# s1 = j (addr)
# s2 = [i]
# s3 = [j]
# s4 = pivot value
# s5 = start address of array (copy of a0)
# s6 = end address of array (copy of a1)
#####
# save return address
        addi sp sp -16
        sw ra 12 sp
```



```

# initialize registers
    addi s0 a0 -4
    addi s1 a1 4
    lw s4 0 a0
    mv s5 a0
    mv s6 a1

loop:
    # scan forward until element >= pivot
    inc_i:
    beq s0 s6 skip_i
    addi s0 s0 4
    lw s2 0 s0
    blt s2 s4 inc_i
    skip_i:
    # state: [i] >= pivot

    # scan backward until element <= pivot
    dec_j:
    beq s1 s5 skip_j
    addi s1 s1 -4
    lw s3 0 s1
    bgt s3 s4 dec_j
    skip_j:
    # state: [j] <= pivot

    # if i >= j return j
    bgeu s0 s1 done_partition

    # We know that [i] > [j]
    # swap(i, j)
    mv a0 s0
    mv a1 s1
    call swap

    j loop

done_partition:
    # restore return address and deallocate stack space
    lw ra 12 sp
    addi sp sp 16

    # return j
    mv a0 s1
    ret

quicksort:
#####
# Perform quicksort on array of word-sized integers
# MUTATES
#
# IN
# a0 = start address
# a1 = end address
#####
    # base case
    # if start address >= end address: return
    bgeu a0 a1 done_quicksort

    # Establish stack frame for recursive quicksort call
    # First, allocate space on stack aligned to 128 bits as per RISC-V ABI
    addi sp sp -16
    sw ra 12 sp
    sw a0 8 sp
    sw a1 4 sp

```

```

    call partition

    # State after returning from partition():
    # a0 = address of pivot (return value of partition())
    # Save this on stack because we are calling quicksort soon
    sw a0 0 sp

    # Sort high array (values greater than pivot)
    # -> quicksort(pivot+1, end)
    addi a0 a0 4
    lw a1 4 sp
    call quicksort

    # Sort low array (values smaller than pivot)
    # -> quicksort(start, pivot)
    lw a0 8 sp
    lw a1 0 sp
    call quicksort

    # Clean up
    lw ra 12 sp
    addi sp sp 16

done_quicksort:
    ret

main:
    # lo = start address of array
    # hi = end address of array
    la a0 lo
    la a1 hi
    call quicksort

```

2.4 Quicksort Stack Frame

Stackframe während der ersten Partition und des ersten Swaps:

quicksort (16 bytes)	
+12	return address -> main
+8	start address of arr (a0 arg)
+4	end address of arr (a1 arg)
partition (16 bytes)	
+12	return address -> quicksort
swap (0 bytes)	

+Offset aus der Sicht des Callees
↑ höhere Adressen ↓ niedrigere Adressen

3. Interrupts

1. Welche Arten von Interrupts gibt es? Wodurch werden diese ausgelöst?

Interrupts sind externe, asynchrone Ausnahmen, die nicht direkt mit dem aktuellen Programmablauf (im Sinne der momentanen Instruktion) zu tun haben (s. 11.17).

Grob gesehen, können wir unterscheiden zwischen IO und Timer Interrupts.

Beispiele für IO:

- Eingaben wie Maus und Tastatur
- IO Prozesse wie Rückmeldung eines Speicherträgers über erfolgreichen Schreibprozess

2. Wie bzw. wann wird auf das Eintreten eines Interrupts reagiert? Wie unterscheiden sich die Arten hier? Auf welcher Betrachtungsebene (Mikroinstruktionen, Instruktionen, Programme, . . .) erfolgt dies?

Control detektiert Ausnahme und leitet Behandlung ein (11.22). Es wird je nach Ausnahme die Ausführung einer Ausnahmeroutine (Interrupt Service Routine, *ISR*) eingeleitet.

Die Arten von Interrupt (IO, Timer) können sich durch Signalart unterscheiden (z.B. sporadische IO Rückmeldung vs. periodischer Timer). Das Schema der Behandlung bleibt aber gleich.

Die Behandlung eines Interrupts erfolgt transparent und ist für das Programm nicht sichtbar – es vergeht lediglich Zeit zwischen zwei Instruktionen. Die *ISR* wird von Control durch Mikroinstruktionen angestoßen.

3. Was ergibt sich daraus für die Verantwortlichkeiten im Falle eines Interrupts? Gehen Sie zudem auf die Sinnhaftigkeit von Rückgabewerten ein.

Bei einem Aufruf einer *ISR* gibt es keinen “Aufrufenden” im klassischen Sinne, da der Aufruf nicht aus einem Programmablauf sondern von der Hardware selbst verursacht wird. Dementsprechend werden die Aufgaben des Aufrufers auf Control sowie die *ISR* selbst verteilt. Control stellt sicher, dass nach der Behandlung des Interrupts das Programm an der selben Stelle fortsetzen kann. Die *ISR* gewährleistet, dass der übrige State erhalten bleibt (Stack, Register).

Dementsprechend haben Interrupt-Routinen keinen Rückgabewert. Es gab keinen Aufrufenden im klassischen Sinne, also auch niemanden der einen Rückgabewert erwarten würde. Das Ziel ist es, das Programm nach dem Interrupt weiter laufen zu lassen, ‘als wäre nichts gewesen’.

4. Es gibt zudem die Möglichkeit, Interrupts zu sperren oder zu priorisieren, sodass die Behandlung nicht umgehend erfolgen muss. Welchen Nutzen könnte dies haben?

Priorisierung und Sperrung von Interrupts könnte vielfältige Anwendung haben:

- **Time-Sharing:** Ermöglicht Multitasking, indem Timer-Interrupts periodisch den Prozess wechseln (Folie 33-34).
- Interrupts Sperren während der Behandlung eines bestehenden Interrupts um Komplexität (nesting) zu vermeiden.
- Nach Interrupt-Quellen unterscheiden – und z.B. zeitkritische Interrupts zuerst behandeln.
- Kritische Programmabläufe ohne Unterbrechung, ‘atomar’, durchlaufen lassen.
- Bei knappen Ressourcen dafür sorgen, dass laufende Programme weiter laufen.

5. Gehen Sie von Interruptbehandlung anhand einer Sprungtabelle aus, von welcher in die richtige Interrupt Service Routine gesprungen wird.

a) Angenommen, die Sprungtabelle befände sich ohne Schreibschutz im Hauptspeicher. Welches Sicherheitsrisiko ergibt sich daraus? Welche Differenzierung wird nötig?

Risiko: Ein Angreifer könnte die Sprungtabelle manipulieren, sodass normaler Code die Kontrolle übernimmt, aber mit erhöhten Rechten läuft (Folie 36).

Differenzierung (Folie 37): Es ist eine "Zwei-Klassen-Gesellschaft" im Prozessor nötig:

1. **Normaler Modus (User Mode):** Eingeschränkte Rechte, kein Schreibzugriff auf Interrupt-Tabelle.
2. **Privilegierter Modus (Supervisor/Kernel Mode):** Darf alles.

Der Wechsel erfolgt automatisch bei einer Ausnahme (Folie 39).

Das Risiko ist privilege escalation. Ein Angreifer könnte durch Veränderung in der Sprungtabelle den Kontrollfluss so umleiten, dass von ihm gesteuerter Code als ISR ausgeführt wird. Da die CPU beim Eintritt einer Ausnahme in den privilegierten Modus wechselt, hätte dieser Code Zugriff auf systemkritische Ressourcen.

Es bleibt sicherzustellen, dass die Sprungtabelle nur zu geprüftem, vertrauenswürdigen Code springt. Es wird also nötig zu differenzieren zwischen systemkritischen (privilegierten) und user-space Operationen. So wird verhindert, dass beliebiger Code in geschützte Speicheradressen schreiben darf.

b) Stellen Sie den Prozess der Ausnahmebehandlung auf eine Ihres Erachtens geeignete Weise dar; schematisch, textuell, . . .

s. 11.28

1. Ausnahme passiert
2. Control detektiert Ausnahme. Kontextsicherung: PC (Fortsetzungsadresse).
3. Passende ISR wird anhand Sprungtabelle herausgesucht.
4. Sprung zur ISR. Kontextsicherung: Register, Stack.
5. Ausführung der ISR im privilegierten Modus.
6. ISR beendet: Kontextwiederherstellung. Rückkehr in nicht-privilegierten Modus.
7. Rücksprung zum gespeicherten PC-Wert.

c) Wie endet die Behandlung einer Ausnahme? Was muss der Prozessor hier leisten?

Es passieren zwei Dinge: privilege de-escalation und Rückkehr zum normalen Programmablauf. Es wird in den Normal-Modus gewechselt, nachdem die ISR im privilegierten Modus ausgeführt wurde (sret). Für die Rückkehr zum normalen Programmablauf stellt der Prozessor sicher, dass die Laufzeitumgebung vollständig auf den Zustand vor dem Eintreten der Ausnahme zurückgesetzt ist. – Dies beinhaltet die Wiederherstellung des Stacks sowie der Register. Das Programm kann dann mithilfe der gespeicherten Fortsetzungsadresse ab der nächsten auszuführenden Instruktion fortfahren.